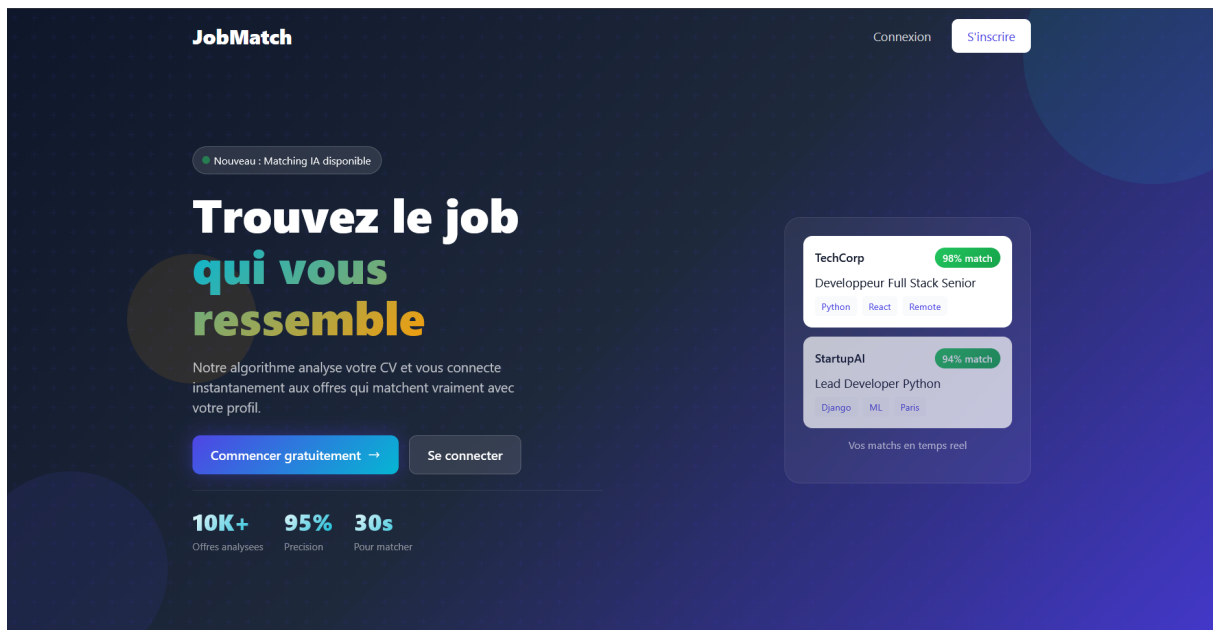


ARTEFACT SCHOOL OF DATA

DATA ENGINEER RNCP37827 BLOC 1

# Rapport de Projet Data Engineer Jobmatch

Pipeline ETL avec Documentation du Code



<https://jobmatch.molp.fr/>

**Équipe :** Mathieu ENGLES, Clément SIRVENTE,  
Mohamed El MADIOUNI, Maxime DICKERSON

**Auteur :** Maxime DICKERSON

**Période :** 22 décembre 2025 - 8 janvier 2026

Session Data Engineering - School Of Data Artefact - Nov. Déc. 2025

# Table des matières

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| Liste des Figures                                                     | 3         |
| Préambule                                                             | 4         |
| <b>1 Vue d'Ensemble Architecturale</b>                                | <b>5</b>  |
| 1.1 Positionnement dans l'Écosystème JobMatch                         | 5         |
| 1.2 Architecture Médaillon : Bronze, Silver, Gold                     | 6         |
| 1.3 Orchestration avec Apache Airflow                                 | 6         |
| <b>2 Collecte : Script d'Extraction Automatisée</b>                   | <b>8</b>  |
| 2.1 Configuration et Déclenchement du DAG                             | 8         |
| 2.2 Configuration des Paramètres par Défaut                           | 8         |
| 2.3 Architecture Docker-in-Docker                                     | 8         |
| 2.4 Authentification OAuth2 avec Cache Intelligent                    | 9         |
| 2.5 Boucle de Collecte Principale et Pagination                       | 9         |
| 2.6 Upload Atomique et Idempotent vers GCS                            | 9         |
| 2.7 Requêtes HTTP avec Rafraîchissement Automatique                   | 10        |
| 2.8 Callback d'Arrêt Automatique de la VM                             | 10        |
| <b>3 Préparation : Nettoyage et Agrégation</b>                        | <b>11</b> |
| 3.1 Vue d'Ensemble de la Transformation Bronze vers Silver            | 11        |
| 3.2 Fonction de Récupération Sécurisée : safe_get                     | 11        |
| 3.3 Normalisation des Timestamps : parse_timestamp                    | 11        |
| 3.4 Transformation Principale : Architecture Multi-Tables             | 12        |
| 3.5 Insertion Parallèle des Tables                                    | 12        |
| 3.6 Transformation Silver vers Gold : Enrichissement Vectoriel        | 12        |
| <b>4 Stockage : Schéma de Base de Données</b>                         | <b>14</b> |
| 4.1 Justification du Choix Technologique                              | 14        |
| 4.2 Modèle Conceptuel de Données (MCD)                                | 14        |
| 4.3 Modèle Logique de Données (MLD)   Couche Silver                   | 15        |
| 4.4 Modèle Logique de Données (MLD)   Couche Gold                     | 16        |
| 4.5 Optimisations de Performance                                      | 16        |
| <b>5 Matching Sémantique : Recommandations d'Emploi Intelligentes</b> | <b>17</b> |
| 5.1 Vue d'Ensemble de l'Architecture de Matching                      | 17        |
| 5.2 Création des Index Vectoriels                                     | 18        |
| 5.3 Le Service VectorSearchService                                    | 18        |
| 5.4 Fonction VECTOR_SEARCH de BigQuery                                | 19        |

|                           |                                                         |           |
|---------------------------|---------------------------------------------------------|-----------|
| 5.5                       | Optimisation par Partition Pruning . . . . .            | 19        |
| 5.6                       | Flux de Matching de Bout en Bout . . . . .              | 20        |
| 5.7                       | Caractéristiques de Performance . . . . .               | 20        |
| <b>Conclusion</b>         |                                                         | <b>21</b> |
| <b>Bibliographie</b>      |                                                         | <b>22</b> |
| <b>A Extraits de Code</b> |                                                         | <b>23</b> |
| A.1                       | Définition du DAG Airflow . . . . .                     | 23        |
| A.2                       | Gestion des Tokens OAuth2 . . . . .                     | 25        |
| A.3                       | Requêtes HTTP avec Auto-Refresh . . . . .               | 26        |
| A.4                       | Upload Atomique GCS . . . . .                           | 26        |
| A.5                       | Boucle de Collecte Principale avec Pagination . . . . . | 27        |
| A.6                       | Parsing de Pagination . . . . .                         | 28        |
| A.7                       | Extraction Sécurisée de Valeurs . . . . .               | 28        |
| A.8                       | Normalisation des Timestamps . . . . .                  | 28        |
| A.9                       | Purge de Partition pour Idempotence . . . . .           | 29        |
| A.10                      | Transformation vers les Tables Silver . . . . .         | 29        |
| A.11                      | Insertion Parallèle des Tables . . . . .                | 30        |
| A.12                      | Génération des Embeddings . . . . .                     | 31        |
| A.13                      | Callback d'Arrêt de VM . . . . .                        | 32        |
| A.14                      | Création des Tables BigQuery (DDL) . . . . .            | 33        |
| A.15                      | Création des Index Vectoriels . . . . .                 | 35        |
| A.16                      | Service de Recherche Vectorielle BigQuery . . . . .     | 36        |
| A.17                      | Structure JSON Exemple . . . . .                        | 39        |

# Table des figures

|     |                                                               |    |
|-----|---------------------------------------------------------------|----|
| 1   | Architecture globale du pipeline ETL JobMatch . . . . .       | 4  |
| 1.1 | Interface utilisateur de la plateforme JobMatch . . . . .     | 5  |
| 1.2 | Architecture Médaillon : Bronze, Silver, Gold . . . . .       | 6  |
| 5.1 | Architecture de matching sémantique de bout en bout . . . . . | 17 |

# Préambule

Ce document fournit une analyse technique complète du module offre-ingestion, pierre angulaire de l'infrastructure de données de la plateforme JobMatch. Ce pipeline ETL (Extract-Transform-Load) automatise l'acquisition quotidienne, la transformation et l'enrichissement sémantique des offres d'emploi depuis l'API France Travail vers un entrepôt de données analytique. L'architecture implémentée suit le paradigme médaillon (Bronze, Silver, Gold) orchestré par Apache Airflow, garantissant traçabilité, reproductibilité et scalabilité.

Ce rapport inclut des extraits de code détaillés en annexes (voir Annexe A) qui illustrent les patterns d'implémentation clés discutés tout au long du document. Chaque référence de code est annotée avec le numéro de listing correspondant pour faciliter les références croisées.

Lien de la plateforme : <https://jobmatch.molp.fr/>

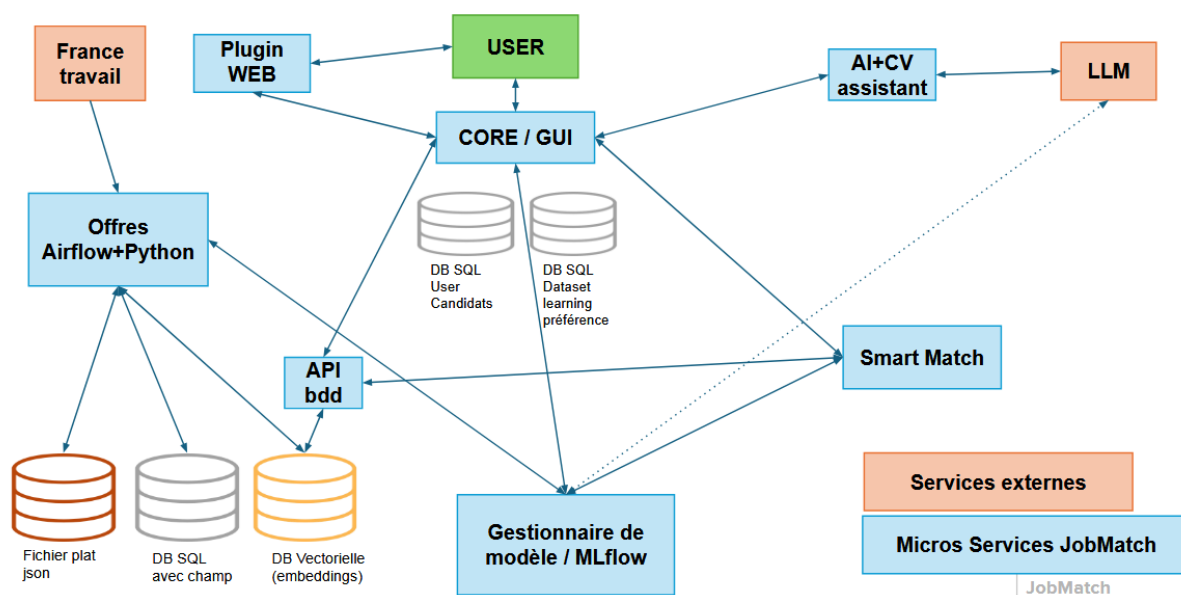


FIGURE 1 – Architecture globale du pipeline ETL JobMatch

# Chapitre 1

## Vue d'Ensemble Architecturale

### 1.1 Positionnement dans l'Écosystème JobMatch

JobMatch est une plateforme de matching intelligent entre candidats et offres d'emploi. Le module offre-ingestion alimente le cœur du système en fournissant un flux quotidien structuré et enrichi d'opportunités professionnelles. Sans ce pipeline, le moteur de recommandation serait privé de sa matière première : les offres d'emploi actualisées du marché français.

Le choix de l'API France Travail comme source primaire n'est pas fortuit. Cette interface programmatique, maintenue par l'opérateur public de l'emploi, agrège les offres de multiples partenaires (agences d'intérim, entreprises, plateformes spécialisées), offrant ainsi une couverture quasi-exhaustive du marché. L'API expose des offres catégorisées selon le référentiel ROME (Répertoire Opérationnel des Métiers et des Emplois), permettant une structuration sémantique native des données.

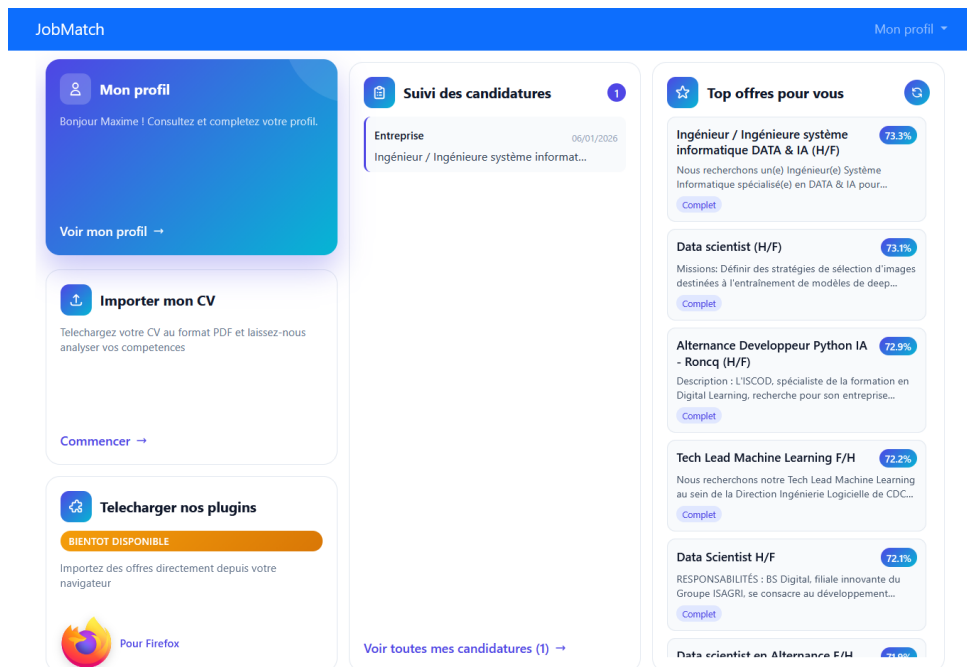


FIGURE 1.1 – Interface utilisateur de la plateforme JobMatch

## 1.2 Architecture Médaille : Bronze, Silver, Gold

L'architecture médaille, popularisée par Databricks, structure le flux de données en trois couches de raffinement croissant :

La **couche Bronze** représente les données brutes, telles qu'extraites de la source. Dans notre implémentation, elle correspond aux fichiers JSON stockés dans Google Cloud Storage (GCS). Ces fichiers préservent l'intégralité du payload API sans transformation, constituant une archive immuable et auditable. La convention de nommage des objets suit le pattern `ingestion_date=YYYY-MM-DD/offer_YYYY-MM-DD.json`, facilitant le partitionnement temporel et la gestion du cycle de vie. Voir le Listing A.17 pour un exemple de structure JSON brute.

La **couche Silver** contient les données nettoyées, normalisées et structurées en tables relationnelles. Le dataset BigQuery `jobmatch_silver` héberge 13 tables interconnectées, représentant une dénormalisation contrôlée du modèle JSON source. Cette couche élimine les incohérences, standardise les formats et établit les relations entre entités.

La **couche Gold** présente les données enrichies optimisées pour la consommation analytique et la recherche sémantique. Le dataset `jobmatch_gold` contient 3 tables intégrant des embeddings vectoriels générés par le modèle `all-MiniLM-L6-v2`. Ces vecteurs de 384 dimensions encodent la sémantique des intitulés et descriptions, permettant une recherche par similarité cosinus. La couche Gold est le fondement de la fonctionnalité de matching intelligent de JobMatch, détaillée au Chapitre 5.

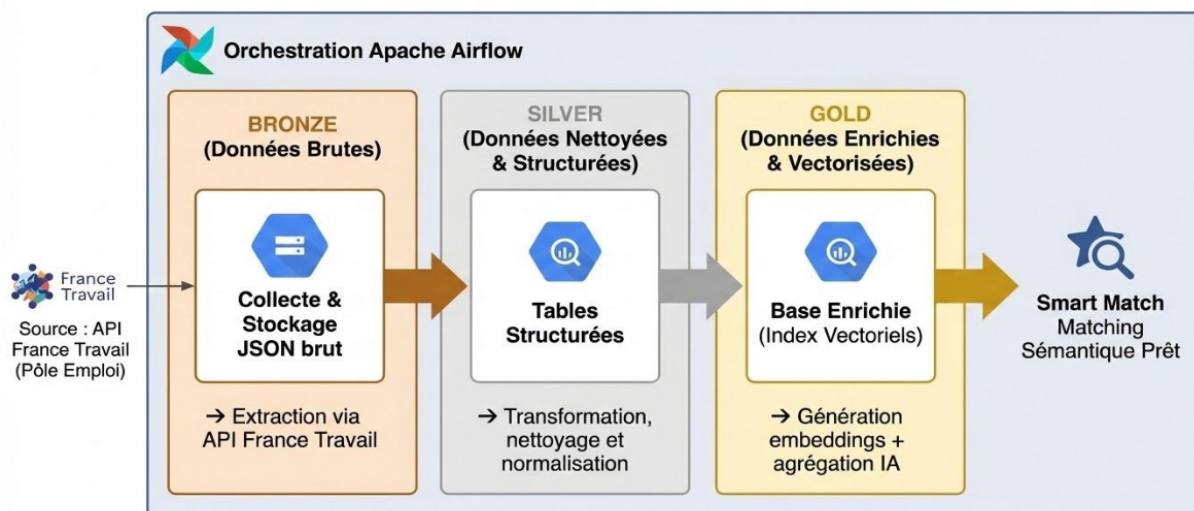


FIGURE 1.2 – Architecture Médaille : Bronze, Silver, Gold

## 1.3 Orchestration avec Apache Airflow

Apache Airflow, framework d'orchestration de workflows développé par Airbnb et maintenu par la Fondation Apache, coordonne l'exécution séquentielle des trois étapes du pipeline. Son modèle déclaratif basé sur les DAG (Directed Acyclic Graphs) exprime

naturellement les dépendances entre tâches, tandis que son architecture distribuée permet une scalabilité horizontale. La définition complète du DAG est fournie dans le Listing [A.1](#).

Le choix d’Airflow par rapport aux alternatives comme Prefect, Dagster ou Luigi repose sur plusieurs critères : maturité de l’écosystème, richesse des opérateurs natifs (notamment `DockerOperator`), interface web de monitoring, et intégration native avec Google Cloud Platform via les providers officiels.



# Chapitre 2

## Collecte : Script d'Extraction Automatisée

### 2.1 Configuration et Déclenchement du DAG

Le fichier `offre_ingestion_dag.py` constitue le point d'entrée de l'orchestration (voir Listing A.1). Il définit un DAG nommé `offre_ingestion_pipeline`, identifiant unique au sein de l'instance Airflow.

La planification temporelle est configurée via le paramètre `schedule_interval` avec la valeur `"57 22 * * *"`, une expression cron signifiant une exécution quotidienne à 22h57. Ce timing n'est pas arbitraire : il cible une fenêtre de faible activité sur l'API France Travail, réduisant les risques de rate-limiting tout en garantissant la disponibilité des offres publiées dans la journée.

Le paramètre `start_date`, fixé au 1er janvier 2026 avec le fuseau horaire Europe/-Paris via `pendulum.datetime`, définit la date de référence pour le scheduler. L'option `catchup=False` désactive le rattrapage automatique des exécutions manquées.

### 2.2 Configuration des Paramètres par Défaut

Le dictionnaire `default_args` centralise la configuration commune à toutes les tâches du DAG. La politique de retry définit le comportement en cas d'échecs transitoires : le paramètre `retries` fixé à 2 permet deux tentatives supplémentaires, avec un délai de 2 minutes entre les tentatives.

Le callback `on_success_callback` pointe vers la fonction `shutdown_vm_on_success` (Listing A.13), un mécanisme d'optimisation des coûts cloud qui arrête automatiquement la VM après la réussite complète du pipeline.

### 2.3 Architecture Docker-in-Docker

L'exécution des scripts Python repose sur `DockerOperator`, un opérateur Airflow qui lance des conteneurs Docker comme tâches du DAG. Cette approche Docker-in-Docker offre plusieurs avantages architecturaux :

- **Isolation des dépendances** : chaque script s'exécute dans un environnement reproductible

- **Portabilité** : le même conteneur fonctionne localement, sur VM GCP ou Kubernetes
- **Contrôle de version** : versions des librairies figées dans `requirements.txt`

## 2.4 Authentification OAuth2 avec Cache Intelligent

L'API France Travail utilise OAuth2 avec le flux `client_credentials`. La fonction `get_token` (Listing A.2) gère l'obtention et le rafraîchissement des tokens d'accès.

Un dictionnaire de cache global stocke le token courant et son timestamp d'expiration. La constante `TOKEN_SKEW_SECONDS` de 60 secondes définit une marge de sécurité : le token est considéré expiré 60 secondes avant son expiration réelle, évitant les rejets durant les requêtes.

La fonction `auth_headers` construit le dictionnaire d'en-têtes HTTP incluant `Authorization: Bearer {token}`.

## 2.5 Boucle de Collecte Principale et Pagination

La fonction `main` (Listing A.5) orchestre l'ensemble du processus d'extraction :

1. Parser la date cible (argument CLI ou veille par défaut)
2. Charger les codes ROME depuis le fichier de configuration
3. Pour chaque code ROME, paginer les résultats API avec un throttle  
`MIN_API_INTERVAL=0.11s`

4. Accumuler toutes les offres en mémoire
5. Upload unique atomique vers GCS

Le parsing de pagination utilise la fonction `extract_total_from_content_range` (Listing A.6) pour parser l'en-tête `Content-Range` de l'API au format « offres 0-149/591250 ».

## 2.6 Upload Atomique et Idempotent vers GCS

La fonction `upload_json_to_gcs_atomic` (Listing A.4) est le cœur de la garantie d'idempotence. Elle utilise le paramètre `if_generation_match=0` qui assure :

- L'upload réussit uniquement si l'objet N'EXISTE PAS
- Les retries du DAG Airflow ne créent pas de doublons
- Les exécutions multiples sont sans effet de bord

## 2.7 Requêtes HTTP avec Rafraîchissement Automatique

La fonction `get_with_auto_refresh` (Listing A.3) encapsule la logique de requête GET avec gestion transparente de l'expiration du token. Si le statut de réponse est 401 (Unauthorized), la fonction déclenche un rafraîchissement forcé et retente automatiquement.

## 2.8 Callback d'Arrêt Automatique de la VM

La fonction `shutdown_vm_on_success` (Listing A.13), attachée au `on_success_callback` du DAG, implémente une optimisation des coûts cloud. Après le succès complet du pipeline, elle déclenche l'arrêt de la VM GCP hébergeant Airflow.

Un délai de 30 secondes permet à Airflow de finaliser les logs. La commande `gcloud compute instances stop` est exécutée en mode détaché.

Cette approche « serverless » réduit drastiquement les coûts : la VM n'est facturée que pendant l'exécution du pipeline.

# Chapitre 3

## Préparation : Nettoyage et Agrégation

### 3.1 Vue d'Ensemble de la Transformation Bronze vers Silver

Le script `transform_offers_to_bigquery_silver.py` implémente la deuxième étape du pipeline : transformer les données JSON brutes (couche Bronze dans GCS) en tables relationnelles structurées (couche Silver dans BigQuery).

Cette transformation accomplit plusieurs objectifs :

- **Nettoyage** : élimine les incohérences (valeurs nulles, chaînes vides)
- **Normalisation** : standardise les types de données pour BigQuery
- **Structuration** : décompose le JSON monolithique en 13 tables relationnelles
- **Enrichissement** : ajoute la colonne `ingestion_date` pour le partitionnement

### 3.2 Fonction de Récupération Sécurisée : `safe_get`

La fonction `safe_get` (Listing A.7) est la pierre angulaire du nettoyage des données. Elle gère trois cas :

- Le dictionnaire est `None` → retourne la valeur par défaut
- La clé est absente → retourne la valeur par défaut
- La valeur est une chaîne vide → retourne la valeur par défaut (important pour BigQuery)

Ce dernier cas unifie le traitement : l'API France Travail retourne parfois des chaînes vides plutôt que `null` pour les champs optionnels.

### 3.3 Normalisation des Timestamps : `parse_timestamp`

La fonction `parse_timestamp` (Listing A.8) convertit les dates ISO 8601 de l'API vers un format compatible BigQuery. L'API utilise le suffixe « Z » pour indiquer UTC, tandis que `datetime.fromisoformat` de Python attend « +00:00 ».

Les exceptions de parsing sont capturées silencieusement, retournant `None` pour les dates malformées. Cette tolérance aux erreurs est délibérée : une date invalide sur une offre ne doit pas bloquer l'ingestion de milliers d'autres offres.

## 3.4 Transformation Principale : Architecture Multi-Tables

La fonction `transform_offers_to_bigquery` (Listing A.10) orchestre la transformation de la liste JSON d’offres en lignes pour les 13 tables BigQuery :

- `offers` : table principale (27 colonnes)
- `offers_lieu_travail` : géolocalisation (1 :1)
- `offers_entreprise` : informations employeur (1 :1)
- `offers_salaire` : informations salariales (0 :1)
- `offers_compétences` : compétences requises (1 :N)
- `offers_ formations` : formations requises (1 :N)
- `offers_langues` : langues requises (1 :N)
- `offers_permis` : permis requis (1 :N)
- `offers_contact` : coordonnées de contact (0 :1)
- `offers_origine` : source de l’offre (0 :1)
- `offers_contexte_travail_horaires` : horaires de travail (1 :N)
- `offers_salaire_complements` : primes salariales (1 :N)
- `offers_qualites_professionnelles` : soft skills (1 :N)

La fonction `delete_existing_partition` (Listing A.9) assure l’idempotence de l’insertion en supprimant les données existantes pour la date cible avant l’insertion. Cela garantit :

- Ré-exécuter le DAG ne crée pas de doublons
- Les données sont toujours fraîches
- Le partitionnement par date limite la portée de l’impact

## 3.5 Insertion Parallèle des Tables

Après l’accumulation des lignes, les jobs d’insertion sont lancés en parallèle via l’API `load_table_from_json` de BigQuery (Listing A.11). La configuration spécifie `WRITE_APPEND` et `ALLOW_FIELD_ADDITION`.

Les jobs sont soumis sans attente immédiate (pattern fire-and-forget), puis une seconde boucle appelle `result()` sur chacun pour attendre la complétion. Cette parallélisation réduit le temps total d’insertion.

## 3.6 Transformation Silver vers Gold : Enrichissement Vectoriel

Le script `transform_offers_to_bigquery_gold.py` implémente la troisième étape du pipeline : enrichir les offres avec des embeddings vectoriels pour la recherche sémantique. Cette transformation est le pont entre les données brutes et le matching intelligent.

**Génération des Embeddings** Le modèle `all-MiniLM-L6-v2` de Sentence-BERT encode la sémantique du texte en vecteurs de 384 dimensions (Listing A.12). Deux types d'embeddings sont générés :

- **Embeddings d'intitulé** : capturent l'essence des titres de poste (ex : « Développeur Python Senior »)
- **Embeddings de description** : encodent la sémantique complète de la description du poste

Le modèle traite les textes par lots de 32 pour l'efficacité mémoire, avec suivi de progression pour le monitoring des jobs longs.

**Structure des Tables Gold** La couche Gold se compose de trois tables :

- `offers` : champs métier dupliqués (id, intitulé, description) pour une récupération rapide sans jointures cross-dataset
- `offers_intitule_embeddings` : vecteurs d'intitulé avec métadonnées (nom du modèle, dimension)
- `offers_description_embeddings` : vecteurs de description avec structure de métadonnées identique

Cette séparation permet des index vectoriels indépendants par table, offrant des stratégies de recherche flexibles. Les embeddings sont stockés comme colonnes `ARRAY<FLOAT64>`, nativement supportées par les fonctions de recherche vectorielle de BigQuery.

# Chapitre 4

## Stockage : Schéma de Base de Données

### 4.1 Justification du Choix Technologique

Le choix de BigQuery comme système de stockage résulte d'une analyse multicritères approfondie :

**Volume et Scalabilité** L'API France Travail expose plusieurs centaines de milliers d'offres quotidiennement. Sur une année, le volume cumulé atteint des dizaines de millions d'enregistrements. BigQuery, en tant que data warehouse serverless, absorbe nativement ces volumes avec son architecture séparant stockage (Colossus) et calcul (Dremel).

**Patterns d'Accès Analytiques** Les requêtes de matching JobMatch suivent des patterns analytiques : agrégations sur millions de lignes, jointures entre tables, filtres multi-dimensionnels. BigQuery excelle avec son moteur de requêtes massivement parallèle.

**Support Natif de la Recherche Vectorielle** BigQuery fournit des fonctions `VECTOR_SEARCH` natives avec support de la distance cosinus, permettant des requêtes k-nearest-neighbor efficaces directement en SQL. Cela élimine le besoin de bases de données vectorielles externes comme Pinecone ou Weaviate, simplifiant l'architecture.

**Modèle Économique** La tarification BigQuery (stockage et facturation des requêtes par volume scanné) correspond au pattern d'usage : ingestion batch quotidienne, requêtes analytiques occasionnelles. Le partitionnement par `ingestion_date` réduit drastiquement les coûts de requête.

**Alternatives Considérées** Elasticsearch aurait excellé pour la recherche textuelle mais aurait compliqué les jointures relationnelles. Apache Cassandra impose un modèle de données rigide. PostgreSQL avec pgvector aurait atteint des limites de scalabilité sur les volumes projetés.

### 4.2 Modèle Conceptuel de Données (MCD)

Le MCD décrit les entités métier et leurs associations, indépendamment de l'implémentation technique.

## Entité Centrale : OFFRE

L'entité OFFRE représente une opportunité d'emploi. Elle possède des attributs intrinsèques : identité (id), contenu (intitule, description), classification (romeCode, romeLibelle), contrat (typeContrat, alternance), conditions (nombrePostes, accessibleTH), qualification (experienceExige), secteur (codeNAF), temporalité (dateCreation, dateActualisation).

### Entités en Relation 1 :1

**LIEU\_TRAVAIL** : localisation géographique. Attributs : adresse, latitude, longitude, code postal, commune.

**ENTREPRISE** : informations employeur. Attributs : nom, description, statut entreprise adaptée.

**SALAIRE** : rémunération proposée. Attributs : fourchette, commentaires, compléments.

**CONTACT** : coordonnées de candidature. Attributs : nom, email, téléphone, URL.

**ORIGINE** : origine de l'offre. Attributs : source, URL, partenaires.

### Entités en Relation N :N avec OFFRE

**COMPETENCE, FORMATION, PERMIS, LANGUE** : attributs spécifiques avec niveau d'exigence (requis/souhaité) le cas échéant.

### Entités complémentaires associées à OFFRE

**QUALITE\_PROFESSIONNELLE** : qualités attendues.

**COMPLEMENT\_SALAIRE** : avantages ou éléments complémentaires de rémunération.

## 4.3 Modèle Logique de Données (MLD) | Couche Silver

Le MLD traduit le MCD en schéma de tables concret pour BigQuery.

### Conventions de Nommage

Les tables utilisent le préfixe `offers_` pour les entités liées. Les clés étrangères sont nommées `offer_id`. La colonne `ingestion_date`, présente dans toutes les tables, sert de clé de partitionnement.

### Table Principale : offers

Structure : id (STRING, NOT NULL), intitule, description, dateCreation (TIMESTAMP), dateActualisation, romeCode, romeLibelle, appellationlibelle, typeContrat,



`typeContratLibelle`, `natureContrat`, `experienceExige`, `experienceLibelle`, `dureeTravailLibelle`, `alternance` (BOOLEAN), `nombrePostes` (INT64), `accessibleTH`, `qualificationCode`, `codeNAF`, `secteurActivite`, `trancheEffectifEtab`, `ingestion_date` (DATE, NOT NULL).

Partitionnement : par jour sur `ingestion_date`. Clustering : `id`, `romeCode`.

Les scripts de création de tables (Listing A.14) définissent les configurations de partitionnement et clustering.

## 4.4 Modèle Logique de Données (MLD) | Couche Gold

Le dataset `jobmatch_gold` contient trois tables optimisées pour le matching sémantique :

### Table offers (Gold)

Structure : `id` (STRING, NOT NULL), `intitule`, `description`, `ingestion_date` (DATE, NOT NULL), `created_at` (TIMESTAMP, NOT NULL).

Partitionnement et clustering identiques à Silver.

### Table offers\_\_intitule\_\_embeddings

Structure : `id`, `intitule_embedded` (ARRAY de FLOAT64, 384 éléments), `embedding_model`, `embedding_dimension` (INT64), `ingestion_date`, `created_at`.

### Table offers\_\_description\_\_embeddings

Structure identique avec `description_embedded`. La séparation permet un index vectoriel par table pour des performances de recherche optimales.

## 4.5 Optimisations de Performance

**Partitionnement Temporel** Le partitionnement par `ingestion_date` divise physiquement les données par jour. Une requête filtrant sur les 7 derniers jours ne scanne que 7 partitions. Sur un historique d'un an, une requête mensuelle ne coûte que  $\frac{30}{365} \approx 8\%$  d'un scan complet.

**Clustering** Le clustering organise physiquement les données au sein de chaque partition. Les requêtes filtrant sur les colonnes clusterisées bénéficient du « block pruning ».

**Colonnes REQUIRED vs NULLABLE** Les colonnes marquées REQUIRED (`id`, `offer_id`, `ingestion_date`) permettent des optimisations internes. D'un point de vue métier, ces colonnes sont effectivement obligatoires.

# Chapitre 5

## Matching Sémantique : Recommandations d'Emploi Intelligentes

Ce chapitre décrit comment les embeddings de la couche Gold alimentent la fonctionnalité de matching intelligent de JobMatch, permettant aux candidats de trouver des offres d'emploi sémantiquement pertinentes au-delà de la simple correspondance par mots-clés.

### 5.1 Vue d'Ensemble de l'Architecture de Matching

La fonctionnalité de matching intelligent transforme le profil ou la requête de recherche d'un candidat dans le même espace vectoriel que les offres d'emploi, puis utilise la recherche vectorielle native de BigQuery pour trouver les offres les plus similaires. L'architecture se compose de trois composants :

1. **Création d'Index Vectoriels** : Configuration unique des index IVF (Inverted File) sur les tables d'embeddings
2. **Embedding de Requête** : Conversion en temps réel de l'entrée utilisateur en vecteur de 384 dimensions
3. **Recherche Vectorielle** : Fonction VECTOR\_SEARCH de BigQuery pour la récupération k-nearest-neighbor

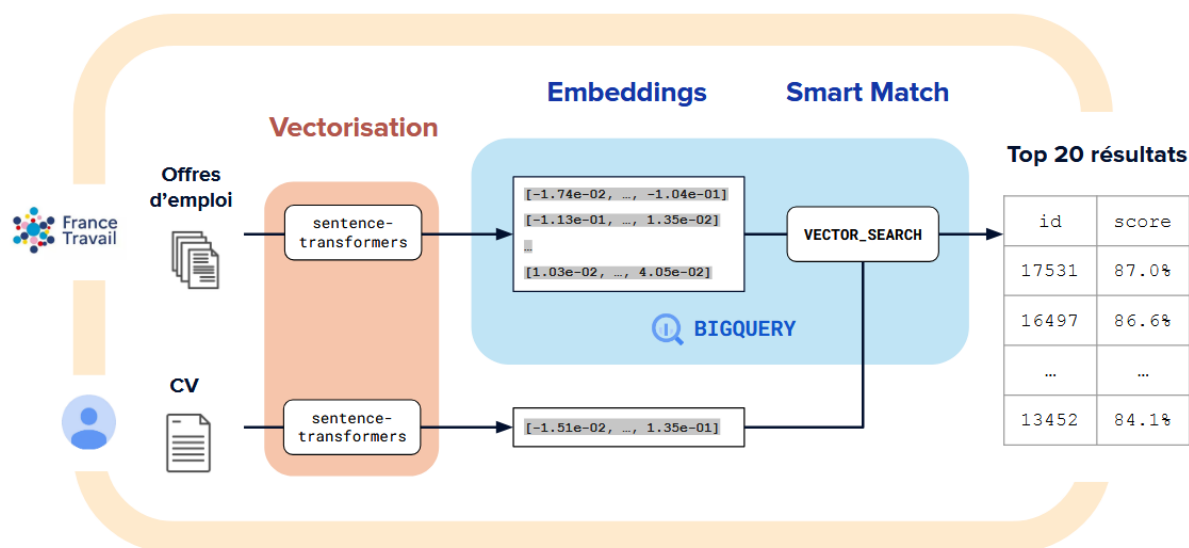


FIGURE 5.1 – Architecture de matching sémantique de bout en bout

## 5.2 Création des Index Vectoriels

BigQuery nécessite des index vectoriels pour une recherche par similarité efficace à grande échelle. Sans index, chaque requête effectuerait un scan brute-force de tous les embeddings. Le script `create_bigquery_gold_vector_indexes.py` (Listing A.15) crée deux index :

- `idx_intitule_embeddings` : Index sur les embeddings d'intitulés de poste
- `idx_description_embeddings` : Index sur les embeddings de descriptions de poste

**Configuration des Index** Les deux index utilisent des paramètres identiques optimisés pour notre cas d'usage :

- `distance_type='COSINE'` : La similarité cosinus est optimale pour les embeddings normalisés de Sentence-BERT, mesurant l'angle entre vecteurs plutôt que la magnitude
- `index_type='IVF'` : L'index Inverted File partitionne l'espace vectoriel en clusters, permettant un temps de recherche sub-linéaire

L'index IVF fournit un excellent compromis entre vitesse de recherche et précision. Pour notre dataset de centaines de milliers d'offres, les requêtes se complètent en millisecondes plutôt qu'en secondes.

**Vérification des Index** Le script inclut une étape de vérification interrogeant `INFORMATION_SCHEMA.VIEWS` pour confirmer la création réussie. Le statut de l'index progresse de `PENDING` à `ACTIVE` pendant que BigQuery optimise les structures de données.

## 5.3 Le Service VectorSearchService

La classe `VectorSearchService` (Listing A.16) encapsule toute la logique de recherche vectorielle avec une interface propre et configurable. Décisions de conception clés :

**Gestion de Configuration** La dataclass `BigQueryConfig` charge tous les noms de tables et colonnes depuis des variables d'environnement, évitant les valeurs en dur. Les variables requises incluent l'ID du projet, l'ID du dataset et les noms de tables. Les variables optionnelles fournissent des valeurs par défaut pour les noms de colonnes.

**Deux Méthodes de Recherche** Le service offre deux méthodes de recherche optimisées pour différents cas d'usage :

1. `find_nearest_embeddings` : Retourne uniquement les IDs et scores de similarité. Pas de surcharge JOIN, optimal pour la production.
2. `find_nearest_embeddings_with_titles` : JOIN avec la table principale pour inclure les titres. Plus coûteux mais utile pour le debug.

## 5.4 Fonction VECTOR\_SEARCH de BigQuery

Le cœur du matching sémantique est la fonction native VECTOR\_SEARCH de BigQuery. La structure de requête SQL (Listing ??) démontre la syntaxe :

```

1 SELECT
2     base.id AS id,
3     1 - (distance / 2) AS similarity,
4     base.ingestion_date
5 FROM VECTOR_SEARCH(
6     TABLE 'project.dataset.embeddings_table',
7     'embedding_column',
8     (SELECT @query_embedding AS embedding_column),
9     top_k => @top_k,
10    distance_type => 'COSINE'
11 )
12 ORDER BY similarity DESC

```

Listing 5.1 – Structure de requête BigQuery VECTOR\_SEARCH

**Calcul du Score de Similarité** Le VECTOR\_SEARCH de BigQuery retourne la *distance* cosinus (0 = identique, 2 = opposé). Nous convertissons cela en score de similarité avec la formule :

$$\text{similarité} = 1 - \frac{\text{distance}}{2}$$

Cela produit des scores dans la plage [0, 1] où 1 indique une similarité parfaite.

**Requêtes Paramétrées** La requête utilise la fonctionnalité de requêtes paramétrées de BigQuery :

- @query\_embedding : Le tableau de 384 floats depuis l'entrée utilisateur
- @top\_k : Nombre de résultats à retourner (configurable par requête)

Cette approche prévient l'injection SQL et permet la mise en cache des plans de requête.

## 5.5 Optimisation par Partition Pruning

Une optimisation critique pour l'efficacité des coûts est le partition pruning lors de la récupération des détails complets d'offre. La méthode `get_full_offer_details` démontre ce pattern :

1. La recherche vectorielle retourne `ingestion_date` aux côtés des IDs d'offre
2. Lors de la récupération des détails complets, ces dates filtrent la requête sur des partitions spécifiques
3. Résultat : réduction des coûts de 100x comparée à un scan complet de table

Sans partition pruning, chaque récupération de détails scannerait la table entière des offres. Avec pruning, seules les partitions journalières pertinentes sont scannées.

## 5.6 Flux de Matching de Bout en Bout

Une requête de matching complète traverse le système comme suit :

1. **Entrée Utilisateur** : Le candidat entre une requête de recherche d'emploi (ex : « data engineer senior Python »)
2. **Génération d'Embedding** : Le même modèle `all-MiniLM-L6-v2` encode la requête en vecteur de 384 dimensions
3. **Recherche Vectorielle** : `find_nearest_embeddings` interroge BigQuery avec `top_k=10`
4. **Traitement des Résultats** : Les scores de similarité sont retournés avec les IDs d'offre
5. **Récupération des Détails** : `get_full_offer_details` récupère les données complètes d'offre avec partition pruning
6. **Réponse** : Offres classées affichées à l'utilisateur avec pourcentages de correspondance

**Embeddings d'Intitulé vs Description** Le paramètre `use_title_embeddings` permet de basculer entre les modes de recherche :

- **Embeddings d'intitulé** (défaut : `False`) : Meilleur pour la similarité sémantique basée sur le contenu du poste
- **Embeddings de description** : Meilleur pour le matching basé sur les descriptions complètes de poste

En pratique, les embeddings de description fournissent un matching plus nuancé car ils capturent les exigences détaillées, tandis que les embeddings d'intitulé sont plus rapides pour le matching de catégories larges.

## 5.7 Caractéristiques de Performance

Les benchmarks sur le dataset de production montrent :

- **Latence recherche vectorielle** : 50-200ms pour les top-10 résultats
- **Octets traités** : 50Mo par requête (optimisé par index)
- **Récupération détails avec partition pruning** : 10-50ms pour 10 offres
- **Sans partition pruning** : 500ms-2s (10-40x plus lent)

L'index IVF réduit la complexité de recherche de  $O(n)$  à approximativement  $O(\sqrt{n})$ , rendant possibles des réponses sub-secondes même avec des millions d'embeddings.

# Conclusion et Perspectives

Le pipeline offre-ingestion incarne les bonnes pratiques de l'ingénierie de données moderne. L'orchestration Airflow assure fiabilité, observabilité et automatisation. L'architecture médaillon structure le flux de données du brut à l'enrichi. Le stockage BigQuery offre scalabilité, performance et intégration écosystème.

Les garanties d'idempotence (préconditions GCS, purge de partition) permettent des ré-exécutions sans risque de duplication. La gestion des erreurs par renouvellement de token, retries Airflow et validation des entrées assure la robustesse face aux défaillances.

Les 13 tables Silver décomposent le JSON monolithique en modèle relationnel requêttable. Les 3 tables Gold ajoutent la dimension sémantique via les embeddings vectoriels. La recherche vectorielle native de BigQuery, alimentée par les index IVF, permet un matching intelligent en millisecondes.

## Améliorations Futures

- Ajout de sources complémentaires (Indeed, LinkedIn)
- Évolution vers le traitement streaming (Kafka, Pub/Sub, Dataflow)
- Enrichissement des embeddings avec des modèles plus puissants (`multilingual-e5-large`)
- Ajout de métriques de qualité des données (Great Expectations, tests dbt)
- Recherche hybride combinant similarité vectorielle et filtres par mots-clés

Le module offre-ingestion démontre qu'une architecture de données bien conçue n'est pas l'apanage des grandes entreprises. Avec les outils cloud modernes, une petite équipe peut construire et opérer un pipeline de grade production, traitant des centaines de milliers d'enregistrements quotidiennement avec fiabilité et efficacité.

**Plateforme accessible en ligne :**

<https://jobmatch.molp.fr/>

# Bibliographie

- [1] Apache Software Foundation. *Documentation Apache Airflow*. <https://airflow.apache.org/docs/>, 2024.
- [2] Google Cloud. *Documentation BigQuery*. <https://cloud.google.com/bigquery/docs>, 2024.
- [3] Google Cloud. *Recherche Vectorielle BigQuery*. <https://cloud.google.com/bigquery/docs/vector-search-intro>, 2024.
- [4] France Travail. *API Offres d'Emploi | Documentation Technique*. <https://francetravail.io/data/api/offres-emploi>, 2024.
- [5] Databricks. *Architecture Médaille*. <https://www.databricks.com/glossary/médaille-architecture>, 2023.
- [6] Reimers, N. et Gurevych, I. *Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks*. Proceedings of EMNLP-IJCNLP, 2019.
- [7] Google Cloud. *Documentation Cloud Storage*. <https://cloud.google.com/storage/docs>, 2024.
- [8] Internet Engineering Task Force. *RFC 6749 : The OAuth 2.0 Authorization Framework*. <https://tools.ietf.org/html/rfc6749>, 2012.

# Annexe A

## Extraits de Code

Cette annexe contient les implémentations de code clés référencées tout au long du rapport.

### A.1 Définition du DAG Airflow

```
1 # Configuration par défaut du DAG
2 default_args = {
3     "owner": "jobmatch-FT-ingestion",
4     "depends_on_past": False,
5     "email_on_failure": True,
6     "email_on_retry": False,
7     "email": [],
8     "retries": 2,
9     "retry_delay": timedelta(minutes=2),
10    "on_success_callback": shutdown_vm_on_success,
11 }
12
13 # Initialisation du DAG
14 with DAG(
15     "offre_ingestion_pipeline",
16     default_args=default_args,
17     description="Pipeline ETL pour ingestion des offres d'emploi France
18     Travail",
19     schedule_interval="57 22 * * *",
20     start_date=pendulum.datetime(2026, 1, 1, tz="Europe/Paris"), # Un
21     jour avant pour execution aujourd'hui
22     catchup=False,
23     tags=["etl", "offre-ingestion", "bigquery"],
24 ) as dag:
25     # Chemin vers les credentials GCP (construit dynamiquement depuis
26     HOST_AIRFLOW_PATH)
27     # Chaque developpeur configure HOST_AIRFLOW_PATH dans son .env local
28     host_airflow_path = os.environ.get("HOST_AIRFLOW_PATH")
29     gcp_credentials_path = f"{host_airflow_path}/credentials/gcp-service
30     -account-key.json"
31
32     # Chemin vers le .env du pipeline offre-ingestion (sur le HOST)
33     offre_ingestion_env_path = f"{host_airflow_path}/../offre-ingestion
34     /.env"
```



```
30
31 # Configuration commune pour tous les operateurs Docker
32 docker_common_config = {
33     "image": "offre-ingestion-pipeline:latest",
34     "api_version": "auto",
35     "auto_remove": True,
36     "docker_url": "unix://var/run/docker.sock",
37     "network_mode": "bridge",
38     "mount_tmp_dir": False,
39     # Montage des volumes necessaires
40     "mounts": [
41         # Credentials GCP (le script les cherche a /app/credentials/
42         # gcp-key.json)
43         {
44             "source": gcp_credentials_path,
45             "target": "/app/credentials/gcp-key.json",
46             "type": "bind",
47             "read_only": True,
48         },
49         # Fichier .env d'offre-ingestion (le script le cherche a /
50         # app/src/.env)
51         # car PROJECT_ROOT = parents[1] depuis /app/src/pipelines/
52         # script.py
53         {"source": offre_ingestion_env_path, "target": "/app/src/.
54         env", "type": "bind", "read_only": True},
55     ],
56     "environment": {"GOOGLE_APPLICATION_CREDENTIALS": "/app/
57     credentials/gcp-key.json"},
58 }
59
60 # Tache 1: Fetch des offres depuis l'API France Travail vers GCS
61 fetch_offers = DockerOperator(
62     task_id="fetch_offers_to_gcs",
63     command=["fetch"],
64     **docker_common_config,
65 )
66
67 # Tache 2: Transformation vers BigQuery Silver (nettoyage et
68 # structuration)
69 transform_to_silver = DockerOperator(
70     task_id="transform_to_bigquery_silver",
71     command=["silver"],
72     **docker_common_config,
73 )
74
75 # Tache 3: Transformation vers BigQuery Gold (embeddings vectoriels)
76 transform_to_gold = DockerOperator(
77     task_id="transform_to_bigquery_gold",
78     command=["gold"],
79     **docker_common_config,
80 )
```

```

75
76 # Definition de l'ordre d'execution des taches
77 fetch_offers >> transform_to_silver >> transform_to_gold

```

Listing A.1 – DAG Airflow pour l'orchestration ETL (offre\_ingestion\_dag.py)

## A.2 Gestion des Tokens OAuth2

```

1 TOKEN_SKEW_SECONDS = 60
2 _token_cache: dict[str, Any] = {"access_token": None, "expires_at": 0.0}
3
4 def _now_epoch() -> float:
5     return time.time()
6
7 def get_token(force_refresh: bool = False) -> str:
8     """
9     R cup re un token valide (cache + refresh).
10    """
11    if (
12        not force_refresh
13        and _token_cache["access_token"]
14        and _now_epoch() < (_token_cache["expires_at"] -
15    TOKEN_SKEW_SECONDS)
16    ):
17        return _token_cache["access_token"]
18
19    data = {
20        "grant_type": "client_credentials",
21        "client_id": CLIENT_ID,
22        "client_secret": CLIENT_SECRET,
23        "scope": SCOPE,
24    }
25
26    r = requests.post(
27        OAUTH_URL,
28        headers={"Content-Type": "application/x-www-form-urlencoded"},
29        data=data,
30        timeout=30,
31    )
32    r.raise_for_status()
33    payload = r.json()
34
35    access_token = payload["access_token"]
36    expires_in = int(payload.get("expires_in", 0))
37
38    _token_cache["access_token"] = access_token
39    _token_cache["expires_at"] = _now_epoch() + expires_in

```

```
40 return access_token
```

Listing A.2 – Authentification OAuth2 avec cache intelligent

## A.3 Requêtes HTTP avec Auto-Refresh

```
1 def get_with_auto_refresh(session: requests.Session, url: str, params:
  dict, timeout: int = 30) -> requests.Response:
2     """
3     Fait un GET.
4     Si 401 -> refresh token -> retente 1 fois.
5     """
6     session.headers.update(auth_headers())
7     r = session.get(url, params=params, timeout=timeout)
8
9     if r.status_code != 401:
10         return r
11
12     get_token(force_refresh=True)
13     session.headers.update(auth_headers())
14     return session.get(url, params=params, timeout=timeout)
```

Listing A.3 – Requête GET avec rafraîchissement automatique du token sur 401

## A.4 Upload Atomique GCS

```
1 def upload_json_to_gcs_atomic(
2     *,
3     bucket_name: str,
4     object_name: str,
5     payload: dict[str, Any],
6     content_type: str = "application/json; charset=utf-8",
7     if_not_exists: bool = True,
8 ) -> str:
9     """
10     Upload JSON vers GCS.
11
12     Atomicité / idempotence:
13     - if_not_exists=True => utilise la pr condition if_generation_match
14     =0 :
15         l'upload choue si l'objet existe déjà, empêchant d' écraser
16         un fichier journalier.
17
18     Retourne le gs:// URL.
19     """
20
21     client = storage.Client(project=GCP_PROJECT_ID)
```

```

20     bucket = client.bucket(bucket_name)
21     blob = bucket.blob(object_name)
22
23     data = json.dumps(payload, ensure_ascii=False).encode("utf-8")
24
25     try:
26         if if_not_exists:
27             blob.upload_from_string(data, content_type=content_type,
28 if_generation_match=0)
29         else:
30             blob.upload_from_string(data, content_type=content_type)
31     except PreconditionFailed as err:
32         # L'objet existe d j (generation != 0)
33         raise RuntimeError(
34             f"Objet GCS d j existant, upload refus (idempotence):
35 gs://{bucket_name}/{object_name}"
36         ) from err
37
38     return f"gs://{bucket_name}/{object_name}"

```

Listing A.4 – Upload atomique et idempotent vers GCS

## A.5 Boucle de Collecte Principale avec Pagination

```

1 def main() -> int:
2     target_date = parse_target_date(sys.argv)
3     rome_codes = load_rome_codes(ROMECODES_PATH)
4
5     session = requests.Session()
6     all_offers: list[dict] = []
7     MIN_API_INTERVAL = 0.11
8
9     for rome in rome_codes:
10         start = 0
11         while True:
12             params = {"codeROME": rome, "range": f"{start}-{start + 149}"
13             }
14
15             r = get_with_auto_refresh(session, api_url, params=params)
16
17             if r.status_code == 204:
18                 break # Pas d'offres pour ce code ROME
19
20             if r.status_code in (200, 206):
21                 offers = r.json().get("resultats") or []
22                 all_offers.extend(offers)
23
24                 if r.status_code == 206: # Contenu Partiel
25                     total = extract_total_from_content_range(
26                         r.headers.get("Content-Range"))

```

```

25         if total and (start + 150) >= total:
26             break
27             start += 150
28             continue
29         break
30
31     upload_json_to_gcs_atomic(bucket_name=GCS_BUCKET, object_name=
32     gcs_object,
33                               payload={"resultats": all_offers})
34
35     return 0

```

Listing A.5 – Boucle de collecte principale avec pagination

## A.6 Parsing de Pagination

```

1 def extract_total_from_content_range(content_range: str | None) -> int |
  None:
2     # Exemple: "offres 0-149/591250" -> 591250
3     if not content_range:
4         return None
5     m = re.search(r"/\s*(\d+)\s*$", content_range)
6     return int(m.group(1)) if m else None

```

Listing A.6 – Parsing de l'en-tête Content-Range

## A.7 Extraction Sécurisée de Valeurs

```

1 def safe_get(obj: dict[str, Any] | None, key: str, default: Any = None)
  -> Any:
2     """R cup re une valeur de mani re s curis e."""
3     if obj is None:
4         return default
5     value = obj.get(key, default)
6     return value if value != "" else default

```

Listing A.7 – Extraction sécurisée de valeur de dictionnaire

## A.8 Normalisation des Timestamps

```

1 def parse_timestamp(ts_str: str | None) -> str | None:
2     """Convertit une date ISO string en format compatible BigQuery."""
3     if not ts_str:
4         return None
5     try:
6         # Parse et reformate pour BigQuery

```

```

7         dt = datetime.fromisoformat(ts_str.replace("Z", "+00:00"))
8         return dt.isoformat()
9     except Exception:
10        return None

```

Listing A.8 – Conversion de timestamp ISO 8601

## A.9 Purge de Partition pour Idempotence

```

1 def delete_existing_partition(client: bigquery.Client, dataset: str,
2   table: str, target_date: date) -> None:
3     """
4     Supprime les lignes de la partition ingestion_date = target_date (
5     idempotence).
6
7     Args:
8         client: Client BigQuery
9         dataset: Nom du dataset
10        table: Nom de la table
11        target_date: Date de la partition      supprimer
12
13    """
14    table_id = f"{GCP_PROJECT_ID}.{dataset}.{table}"
15    query = f"DELETE FROM '{table_id}' WHERE ingestion_date =
16    @target_date" # nosec B608
17    job_config = bigquery.QueryJobConfig(
18        query_parameters=[bigquery.ScalarQueryParameter("target_date", "
19    DATE", target_date)]
20    )
21    client.query(query, job_config=job_config).result()

```

Listing A.9 – Suppression de partition pour idempotence

## A.10 Transformation vers les Tables Silver

```

1 def transform_offers_to_bigquery(offers: list[dict], target_date: date):
2     """Transforme les offres JSON en tables BigQuery normalis es."""
3     rows_offers, rows_lieu, rows_compétences = [], [], []
4
5     for offer in offers:
6         offer_id = offer.get("id", "")
7
8         rows_offers.append({
9             "id": offer_id,
10            "intitule": offer.get("intitule"),
11            "description": offer.get("description"),
12            "dateCreation": parse_timestamp(offer.get("dateCreation")),
13            "ingestion_date": target_date.isoformat(),

```

```

14     })
15
16     lieu = offer.get("lieuTravail")
17     if lieu:
18         rows_lieu.append({
19             "offer_id": offer_id,
20             "libelle": safe_get(lieu, "libelle"),
21             "latitude": safe_get(lieu, "latitude"),
22             "ingestion_date": target_date.isoformat(),
23         })
24
25     for comp in offer.get("competences", []):
26         rows_competences.append({
27             "offer_id": offer_id,
28             "libelle": safe_get(comp, "libelle"),
29             "exigence": safe_get(comp, "exigence"),
30             "ingestion_date": target_date.isoformat(),
31         })

```

Listing A.10 – Transformation JSON vers BigQuery (extrait)

## A.11 Insertion Parallèle des Tables

```

1  jobs: list[tuple[str, bigquery.LoadJob, int]] = []
2
3  # 1) Lancer tous les jobs (sans attendre)
4  for table_name, rows in tables_data.items():
5      if not rows:
6          print(f" {table_name:45s} : 0 lignes (ignor )")
7          continue
8
9      table_id = f"{GCP_PROJECT_ID}.{DATASET_ID}.{table_name}"
10
11     job_config = bigquery.LoadJobConfig(
12         write_disposition=bigquery.WriteDisposition.WRITE_APPEND,
13         schema_update_options=[bigquery.SchemaUpdateOption.
14     ALLOW_FIELD_ADDITION],
15     )
16
17     job = client.load_table_from_json(rows, table_id, job_config=
18     job_config)
19     jobs.append((table_name, job, len(rows)))
20     print(f" {table_name:45s} : job lanc ({len(rows):6d} lignes)"
21 )
22
23 print("-" * 80)
24
25 # 2) Attendre la fin de tous les jobs
26 for table_name, job, nrows in jobs:

```

```

24     job.result() # l ve une exception si le job choue
25     stats[table_name] = nrows
26     print(f" {table_name:45s} : {nrows:6d} lignes (termin )")
27
28     print("-" * 80)
29     return stats

```

Listing A.11 – Jobs d’insertion BigQuery en parallèle

## A.12 Génération des Embeddings

```

1 from shared.embeddings.providers import
  create_sentence_transformers_embedder
2
3 def generate_embeddings(offers: list[dict[str, Any]]) -> tuple[list[str
  ], np.ndarray, np.ndarray]:
4     """
5     G n re les embeddings pour les intitul s et descriptions.
6
7     Args:
8         offers: Liste des offres avec id, intitul e, description
9
10    Returns:
11        Tuple (ids, intitul es_embeddings, descriptions_embeddings)
12    """
13    if not offers:
14        return [], np.array([]), np.array([])
15
16    print(f"\nInitialisation du mod le d'embedding : {EMBEDDING_MODEL}"
17    )
18    embedder = create_sentence_transformers_embedder(
19        model=EMBEDDING_MODEL,
20        device="cpu",
21        batch_size=BATCH_SIZE,
22        normalize=NORMALIZE,
23    )
24
25    # Pr parer les donn es
26    ids = [offer["id"] for offer in offers]
27    intitul es = [offer["intitule"] for offer in offers]
28    descriptions = [offer["description"] for offer in offers]
29
30    # G n rer les embeddings par batch
31    print(f"G n ration des embeddings pour {len(offers)} offres...")
32    print(f" - Mod le: {EMBEDDING_MODEL}")
33    print(f" - Batch size: {BATCH_SIZE}")
34    print(f" - Normalisation: {NORMALIZE}")
35
36    all_texts = intitul es + descriptions

```



```

36     all_embeddings = embedder(all_texts)
37
38     n = len(offers)
39     intitules_embeddings = all_embeddings[:n]
40     descriptions_embeddings = all_embeddings[n : 2 * n]
41
42     print("    Embeddings g n r s:")
43     print(f"    - Intitul s: {intitules_embeddings.shape}")
44     print(f"    - Descriptions: {descriptions_embeddings.shape}")
45
46     return ids, intitules_embeddings, descriptions_embeddings

```

Listing A.12 – Génération des embeddings vectoriels pour recherche sémantique

## A.13 Callback d’Arrêt de VM

```

1 def shutdown_vm_on_success(context):
2     """
3     Callback ex cut apr s le succ s complet du DAG.
4     Arr te la VM GCP avec un d lai de 30 secondes pour permettre
5     Airflow de finaliser l' criture des statuts et logs.
6     """
7     vm_name = os.environ.get("GCP_VM_NAME")
8     zone = os.environ.get("GCP_VM_ZONE", "europe-west9-b")
9
10    if not vm_name:
11        print("    GCP_VM_NAME non d fini , impossible d'arr ter la
12        VM")
13        return
14
15    print(f"    Pipeline termin avec succ s ! Arr t de la VM {
16    vm_name} dans 30 secondes...")
17
18    # Authentification avec le service account et arr t de la VM
19    cmd = f"""sleep 30 && \
20        gcloud auth activate-service-account --key-file=
21        $GOOGLE_APPLICATION_CREDENTIALS && \
22        gcloud compute instances stop {vm_name} --zone={zone}"""
23
24    # Lancer le shutdown en arri re-plan (d tach ) pour que le
25    callback se termine proprement
26    subprocess.Popen(["bash", "-c", cmd], stdout=subprocess.DEVNULL,
27    stderr=subprocess.DEVNULL)
28    print("    Commande d'arr t GCP programm e.")

```

Listing A.13 – Arrêt automatique de VM après succès du DAG

## A.14 Création des Tables BigQuery (DDL)

```

1 # D f i n i t i o n   d e s   s c h e m a s   p o u r   l e s   13   t a b l e s
2 tables_schemas = {
3     "offers": [
4         bigquery.SchemaField("id", "STRING", mode="REQUIRED"),
5         bigquery.SchemaField("intitule", "STRING"),
6         bigquery.SchemaField("description", "STRING"),
7         bigquery.SchemaField("dateCreation", "TIMESTAMP"),
8         bigquery.SchemaField("dateActualisation", "TIMESTAMP"),
9         bigquery.SchemaField("romeCode", "STRING"),
10        bigquery.SchemaField("romeLibelle", "STRING"),
11        bigquery.SchemaField("appellationlibelle", "STRING"),
12        bigquery.SchemaField("typeContrat", "STRING"),
13        bigquery.SchemaField("typeContratLibelle", "STRING"),
14        bigquery.SchemaField("natureContrat", "STRING"),
15        bigquery.SchemaField("experienceExige", "STRING"),
16        bigquery.SchemaField("experienceLibelle", "STRING"),
17        bigquery.SchemaField("dureeTravailLibelle", "STRING"),
18        bigquery.SchemaField("dureeTravailLibelleConverti", "STRING"),
19        bigquery.SchemaField("alternance", "BOOLEAN"),
20        bigquery.SchemaField("nombrePostes", "INT64"),
21        bigquery.SchemaField("accessibleTH", "BOOLEAN"),
22        bigquery.SchemaField("qualificationCode", "STRING"),
23        bigquery.SchemaField("qualificationLibelle", "STRING"),
24        bigquery.SchemaField("codeNAF", "STRING"),
25        bigquery.SchemaField("secteurActivite", "STRING"),
26        bigquery.SchemaField("secteurActiviteLibelle", "STRING"),
27        bigquery.SchemaField("trancheEffectifEtab", "STRING"),
28        bigquery.SchemaField("offresManqueCandidats", "BOOLEAN"),
29        bigquery.SchemaField("entrepriseAdaptee", "BOOLEAN"),
30        bigquery.SchemaField("employeurHandiEngage", "BOOLEAN"),
31        bigquery.SchemaField("ingestion_date", "DATE", mode="REQUIRED"),
32    ],
33    "offers_lieu_travail": [
34        bigquery.SchemaField("offer_id", "STRING", mode="REQUIRED"),
35        bigquery.SchemaField("libelle", "STRING"),
36        bigquery.SchemaField("latitude", "FLOAT64"),
37        bigquery.SchemaField("longitude", "FLOAT64"),
38        bigquery.SchemaField("codePostal", "STRING"),
39        bigquery.SchemaField("commune", "STRING"),
40        bigquery.SchemaField("ingestion_date", "DATE", mode="REQUIRED"),
41    ],
42    "offers_entreprise": [
43        bigquery.SchemaField("offer_id", "STRING", mode="REQUIRED"),
44        bigquery.SchemaField("nom", "STRING"),
45        bigquery.SchemaField("description", "STRING"),
46        bigquery.SchemaField("entrepriseAdaptee", "BOOLEAN"),
47        bigquery.SchemaField("ingestion_date", "DATE", mode="REQUIRED"),
48    ],

```

```
49     "offers_salaire": [  
50         bigquery.SchemaField("offer_id", "STRING", mode="REQUIRED"),  
51         bigquery.SchemaField("libelle", "STRING"),  
52         bigquery.SchemaField("commentaire", "STRING"),  
53         bigquery.SchemaField("complement1", "STRING"),  
54         bigquery.SchemaField("complement2", "STRING"),  
55         bigquery.SchemaField("ingestion_date", "DATE", mode="REQUIRED"),  
56     ],  
57     "offers_salaire_complements": [  
58         bigquery.SchemaField("offer_id", "STRING", mode="REQUIRED"),  
59         bigquery.SchemaField("code", "STRING"),  
60         bigquery.SchemaField("libelle", "STRING"),  
61         bigquery.SchemaField("ingestion_date", "DATE", mode="REQUIRED"),  
62     ],  
63     "offers_competences": [  
64         bigquery.SchemaField("offer_id", "STRING", mode="REQUIRED"),  
65         bigquery.SchemaField("code", "STRING"),  
66         bigquery.SchemaField("libelle", "STRING"),  
67         bigquery.SchemaField("exigence", "STRING"),  
68         bigquery.SchemaField("ingestion_date", "DATE", mode="REQUIRED"),  
69     ],  
70     "offers_qualites_professionnelles": [  
71         bigquery.SchemaField("offer_id", "STRING", mode="REQUIRED"),  
72         bigquery.SchemaField("libelle", "STRING"),  
73         bigquery.SchemaField("description", "STRING"),  
74         bigquery.SchemaField("ingestion_date", "DATE", mode="REQUIRED"),  
75     ],  
76     "offers_formationen": [  
77         bigquery.SchemaField("offer_id", "STRING", mode="REQUIRED"),  
78         bigquery.SchemaField("codeFormation", "STRING"),  
79         bigquery.SchemaField("domaineLibelle", "STRING"),  
80         bigquery.SchemaField("niveauLibelle", "STRING"),  
81         bigquery.SchemaField("commentaire", "STRING"),  
82         bigquery.SchemaField("exigence", "STRING"),  
83         bigquery.SchemaField("ingestion_date", "DATE", mode="REQUIRED"),  
84     ],  
85     "offers_permis": [  
86         bigquery.SchemaField("offer_id", "STRING", mode="REQUIRED"),  
87         bigquery.SchemaField("libelle", "STRING"),  
88         bigquery.SchemaField("exigence", "STRING"),  
89         bigquery.SchemaField("ingestion_date", "DATE", mode="REQUIRED"),  
90     ],  
91     "offers_langues": [  
92         bigquery.SchemaField("offer_id", "STRING", mode="REQUIRED"),  
93         bigquery.SchemaField("libelle", "STRING"),  
94         bigquery.SchemaField("exigence", "STRING"),  
95         bigquery.SchemaField("ingestion_date", "DATE", mode="REQUIRED"),  
96     ],  
97     "offers_contact": [  
98         bigquery.SchemaField("offer_id", "STRING", mode="REQUIRED"),  
99         bigquery.SchemaField("nom", "STRING"),
```

```

100     bigquery.SchemaField("coordonnees1", "STRING"),
101     bigquery.SchemaField("coordonnees2", "STRING"),
102     bigquery.SchemaField("coordonnees3", "STRING"),
103     bigquery.SchemaField("courriel", "STRING"),
104     bigquery.SchemaField("telephone", "STRING"),
105     bigquery.SchemaField("urlRecruteur", "STRING"),
106     bigquery.SchemaField("commentaire", "STRING"),
107     bigquery.SchemaField("ingestion_date", "DATE", mode="REQUIRED"),
108 ],
109 "offers_origine": [
110     bigquery.SchemaField("offer_id", "STRING", mode="REQUIRED"),
111     bigquery.SchemaField("origine", "STRING"),
112     bigquery.SchemaField("urlOrigine", "STRING"),
113     bigquery.SchemaField("partenaires", "STRING"),
114     bigquery.SchemaField("ingestion_date", "DATE", mode="REQUIRED"),
115 ],
116 "offers_contexte_travail_horaires": [
117     bigquery.SchemaField("offer_id", "STRING", mode="REQUIRED"),
118     bigquery.SchemaField("horaire", "STRING"),
119     bigquery.SchemaField("ingestion_date", "DATE", mode="REQUIRED"),
120 ],
121 }

```

Listing A.14 – Création du schéma BigQuery Silver

## A.15 Création des Index Vectoriels

```

1 vector_indexes = [
2     {
3         "name": "idx_intitule_embeddings",
4         "table": TABLE_TITLE,
5         "column": "intitule_embedded",
6         "description": "Index vectoriel sur les embeddings des
intitulés",
7     },
8     {
9         "name": "idx_description_embeddings",
10        "table": TABLE_DESC,
11        "column": "description_embedded",
12        "description": "Index vectoriel sur les embeddings des
descriptions",
13    },
14 ]
15
16 print("\n[2/3] Création des index vectoriels...")
17 print("-" * 80)
18
19 for idx in vector_indexes:
20     idx_name = idx["name"]

```

```

21 table_fq = f"{PROJECT_ID}.{DATASET_ID}.{idx['table']}"
22 idx_column = idx["column"]
23
24 # SQL pour cr er l'index vectoriel
25 create_index_query = f"""
26 CREATE VECTOR INDEX IF NOT EXISTS {idx_name}
27 ON '{table_fq}'({idx_column})
28 OPTIONS(
29     distance_type='COSINE',
30     index_type='IVF'
31 )
32 """
33
34 try:
35     print(f"\nCr ation de l'index: {idx_name}")
36     print(f" - Table: {table_fq}")
37     print(f" - Colonne: {idx_column}")
38     print(" - Type de distance: COSINE")
39     print(" - Type d'index: IVF")
40     print(f" - Description: {idx['description']}")
41
42     job = client.query(create_index_query, location=LOCATION)
43     job.result()
44
45     print(f"      Index cr      avec succ s : {idx_name}")
46
47 except Exception as e:
48     error_msg = str(e).strip()
49     # Important: ici "Already Exists" peut aussi signifier
50     # contrainte 1 index/table
51     # mais avec notre architecture (1 index par table) a ne
52     # devrait pas arriver,
53     # sauf si tu relances le script ou si le nom existe d j dans
54     # le dataset.
55     if "already exists" in error_msg.lower():
56         print(f"      Index d j      existant (ou conflit) : {idx_name}\n
nerror : {error_msg}")
57     else:
58         print(f"      Erreur lors de la cr ation de {idx_name}: {
59 error_msg}")
60         raise

```

Listing A.15 – Création des index vectoriels BigQuery Gold

## A.16 Service de Recherche Vectorielle BigQuery

```

1 class VectorSearchService:
2     """Service for performing vector searches with BigQuery."""
3

```

```

4     def __init__(self, config: BigQueryConfig = None):
5         """Initialize with config from environment if not provided."""
6         self.config = config or BigQueryConfig.from_env()
7         self.client = bigquery.Client(project=self.config.project_id)
8
9     def find_nearest_embeddings(
10         self,
11         query_embedding: list[float],
12         top_k: int = 10,
13         query_id: str = None,
14         query_metadata: dict = None,
15         use_title_embeddings: bool = False,
16     ) -> list[dict]:
17         """
18         Find top-k nearest embeddings WITHOUT joining with main table.
19         Returns similarity scores (0-1 range, higher is better).
20
21         This is the recommended method for production use - no JOIN
22         overhead.
23
24         Args:
25             query_embedding: The embedding vector to search with
26             top_k: Number of top results to return
27             query_id: Optional ID for logging
28             query_metadata: Optional metadata for logging
29             use_title_embeddings: If False (default), use description
30             embeddings; if True, use title embeddings
31
32         Returns:
33             List of dicts with keys: id, similarity, ingestion_date
34         """
35         start_time = datetime.now()
36         query_id = query_id or f"search_{start_time.strftime('%Y%m%d_%H%M%S')}"
37
38         # Select which embeddings table to use
39         embeddings_table = (
40             self.config.title_embeddings_table if use_title_embeddings
41         else self.config.description_embeddings_table
42         )
43
44         # Log input
45         logger.info(f"Starting vector search (no JOIN) - Query ID: {
46             query_id}")
47         logger.info(f"Parameters: top_k={top_k}, embedding_dim={len(
48             query_embedding)}")
49         logger.info(f"Using embeddings table: {embeddings_table}")
50         if query_metadata:
51             logger.info(f"Metadata: {json.dumps(query_metadata)}")
52
53         # Build optimized query - NO JOIN, just embeddings table with

```

```

ingestion_date
49     query = f"""
50     SELECT
51         base.{self.config.id_column} AS id,
52         1 - (distance / 2) AS similarity,
53         base.{self.config.ingestion_date_column} AS ingestion_date
54     FROM VECTOR_SEARCH(
55         TABLE {self.config.get_table_ref(embeddings_table)},
56         '{self.config.embedding_column}',
57         (SELECT @query_embedding AS {self.config.embedding_column}),
58         top_k => @top_k,
59         distance_type => 'COSINE'
60     )
61     ORDER BY similarity DESC
62     """ # nsec B608
63
64     job_config = bigquery.QueryJobConfig(
65         query_parameters=[
66             bigquery.ArrayQueryParameter("query_embedding", "FLOAT64", query_embedding),
67             bigquery.ScalarQueryParameter("top_k", "INT64", top_k),
68         ]
69     )
70
71     try:
72         logger.info("Executing BigQuery vector search (no JOIN)...")
73         query_job = self.client.query(query, job_config=job_config)
74         results = query_job.result()
75
76         # Log query statistics
77         logger.info(f"Query completed - Bytes processed: {query_job.total_bytes_processed}")
78         logger.info(f"Query duration: {query_job.ended - query_job.started}")
79
80         # Process results
81         matches = []
82         for row in results:
83             matches.append(
84                 {
85                     "id": row.id,
86                     "similarity": float(row.similarity),
87                     "ingestion_date": row.ingestion_date.isoformat()
88                 }
89             )
90
91         # Log results summary
92         end_time = datetime.now()
93         duration = (end_time - start_time).total_seconds()
94

```

```

95         logger.info(f"Search completed successfully in {duration:.2f
    }s")
96         logger.info(f"Found {len(matches)} matches")
97
98         # Log top results
99         logger.info("Top results:")
100         for i, match in enumerate(matches[: min(5, len(matches))],
    1):
101             logger.info(f"    {i}. ID: {match['id']}, Similarity: {
    match['similarity']:.4f}")
102
103         # Log all results
104         result_summary = [{"id": m["id"], "similarity": f"{m['
    similarity']:.4f}"} for m in matches]
105         logger.info(f"All results: {json.dumps(result_summary)}")
106
107         return matches
108
109     except Exception as e:
110         logger.error(f"Vector search failed: {str(e)}", exc_info=
    True)
111         raise

```

Listing A.16 – Service de recherche vectorielle pour matching sémantique

## A.17 Structure JSON Exemple

```

1 {
2     "id": "048KLTP",
3     "intitule": "Boulang er / Boulang re (H/F)",
4     "description": "Nous recherchons un/e Boulang er / re pour notre nouveau
    magasin.",
5     "dateCreation": "2022-10-23T08:15:42Z",
6     "dateActualisation": "2022-10-23T08:15:42Z",
7     "lieuTravail": {
8         "libelle": "74 - ANNECY",
9         "latitude": 45.901584,
10        "longitude": 6.125296,
11        "codePostal": "74000",
12        "commune": "74010"
13    },
14    "romeCode": "D1102",
15    "romeLibelle": "Boulang er / Boulang re",
16    "appellationlibelle": "Boulang er / Boulang re",
17    "entreprise": {
18        "nom": "Le boulang er austral",
19        "description": "Votre nouvelle boulangerie locale",
20        "logo": "https://boulang er-austral.net/logo.png",
21        "url": "https://boulang er-austral.net",

```



```
22     "entrepriseAdaptee": true
23 },
24 "typeContrat": "CDD",
25 "typeContratLibelle": "CDD - 6 Mois",
26 "natureContrat": "Contrat travail",
27 "experienceExige": "D",
28 "experienceLibelle": "D butant accept ",
29 "experienceCommentaire": "Exp rience dans la vente souhait e",
30 "formations": [
31     {
32         "codeFormation": "21538",
33         "domaineLibelle": "boulangerie",
34         "niveauLibelle": "CAP, BEP et  quivalents ",
35         "commentaire": "Mention bien souhait e",
36         "exigence": "E"
37     }
38 ],
39 "langues": [
40     {
41         "libelle": "Anglais",
42         "exigence": "E"
43     }
44 ],
45 "permis": [
46     {
47         "libelle": "B - V hicule l ger",
48         "exigence": "S"
49     }
50 ],
51 "outilsBureautiques": "Traitement de texte",
52 "competences": [
53     {
54         "code": "483320",
55         "libelle": "Maintenir un environnement de travail propre et
organis ",
56         "exigence": "E"
57     }
58 ],
59 "salaire": {
60     "libelle": "Mensuel de 1923.00 Euros sur 12 mois",
61     "commentaire": "Selon exp rience",
62     "complement1": "V hicule de fonction",
63     "complement2": "Prime de vacances",
64     "listeComplements": "[V hicule de fonction, Prime de vacances, ...]
"
65 },
66 "dureeTravailLibelle": "35H",
67 "dureeTravailLibelleConverti": "Temps plein",
68 "complementExercice": "Gestion de la comptabilit ",
69 "conditionExercice": "Travail de nuit",
70 "alternance": false,
```

```
71  "contact": {
72    "nom": "Etienne Dupont",
73    "coordonnees1": "12 impasse du caillou",
74    "coordonnees2": "string",
75    "coordonnees3": "string",
76    "telephone": "06 12 34 56 78",
77    "courriel": "Pour postuler, utiliser le lien suivant : https://
candidat.francetravail.fr/offres/recherche/detail/XXXXXXX",
78    "commentaire": "A contacter apr s 19h",
79    "urlRecruteur": "https://boulanger-austral.net",
80    "urlPostulation": "https://boulanger-austral.net/carrieres"
81  },
82  "agence": {
83    "telephone": "06 12 34 56 78",
84    "courriel": "Pour postuler, utiliser le lien suivant : https://
candidat.francetravail.fr/offres/recherche/detail/XXXXXXX"
85  },
86  "nombrePostes": 3,
87  "accessibleTH": true,
88  "deplacementCode": "1",
89  "deplacementLibelle": "Jamais",
90  "qualificationCode": "6",
91  "qualificationLibelle": "Employ qualifi ",
92  "codeNAF": "10.71C",
93  "secteurActivite": "10",
94  "secteurActiviteLibelle": "Boulangerie et boulangerie-p tisserie",
95  "qualitesProfessionnelles": [
96    {
97      "libelle": "Faire preuve d'autonomie",
98      "description": "Capacit prendre en charge son activit sans
devoir tre encadr de fa on continue."
99    }
100  ],
101  "trancheEffectifEtab": "1 5 employ s",
102  "origineOffre": {
103    "origine": "2",
104    "urlOrigine": "https://partenaire-offre.net/boulanger-austral-46841"
105  },
106  "partenaires": [
107    {
108      "nom": "PARTENAIRE1",
109      "url": "https://partenaire-offre.net/boulanger-austral-46841",
110      "logo": "https://partenaire-offre.net/logo.png"
111    }
112  ],
113  "offresManqueCandidats": true,
114  "contexteTravail": {
115    "horaires": "35H Travail le samedi",
116    "conditionsExercice": "Port de tenue professionnelle ou d'uniforme"
117  },
```

```
118 "entrepriseAdaptee": true,  
119 "employeurHandiEngage": true  
120 }
```

Listing A.17 – Structure JSON brute de l'API France Travail